DLA - Lab. 1

Lapo Bisognin

1 Introduction

1.1 Settings

All the following experiments and development (e.g., training, testing) were conducted on the same hardware, which includes an Nvidia RTX 3070 GPU, an Intel i9-10900 CPU (2.9 GHz), and 32 GB of RAM. The primary Python libraries utilized are Torch and Scikit-learn. The virtual environment was managed using uv.

1.2 Use of LLMs

Within the scope of this laboratory, LLMs were used as an aid for translating texts into English and for looking up specific functions/attributes in the libraries employed in the project. A more extensive use was made to define functions for plotting curves, such as utils.plot_pr_curves.

2 Exercise 1

2.1 Exercise 1.1

In the preliminary study of the GTSRB dataset, the class distribution and its cardinality were analyzed to assess potential imbalances regarding the number of samples per category. A severe disproportion emerges between the most frequent class (class 1) and the least frequent one (class 0), with the former exhibiting a sample size ten times greater than the latter. Furthermore, the class distribution histogram clearly indicates a pronounced imbalance across all categories. Consequently, the evaluation methodologies adopted in the subsequent experiments must inherently account for this data distribution property.

Moreover, by visually identifying the dataset classes, specific a priori assumptions can be formulated regarding the upcoming experimental results. Within the 43 categories, distinct subsets of signs can be clustered based on geometric and graphical similarities: [0–8] speed limits, [9–11, 15–17] prohibitions, [18–31] danger/warning signs, [32–40] mandatory directions, and [41, 42] end-of-restriction signs. Conversely, classes [12, 13, 14] do not fall into any of these predefined macro-categories.

2.2 Exercise 1.2

Dopo aver definito una pipeline capace di definire il backbone baseline atto alla computazione delle feature si passano queste al classificatore SVC su cui, dopo essere fittato sulle feature di training, si valutano le performance sul dataset di testing evidenziando valori accettabili.

Table 1: Classification report (GTSRB).

	Precision	**Recall**	**F1-score**	**Support**
Accuracy			0.79	12630
Macro avg	0.71	0.75	0.72	12630
Weighted avg	0.80	0.79	0.79	12630

2.3 Exercise 1.3

After developing a set of functions within the `model_builder` module to automatically identify the model's classification head and replace it with either a Linear or an MLP head—configured with 43 output nodes to match the dataset classes—an empirical evaluation was conducted. Specifically, accuracy and loss metrics were monitored and compared during the training, validation, and testing phases across various backbone architectures, strictly varying the type of final classification head and the degree of layer freezing.

The experimental matrix compared three distinct backbones (`resnet18`, `resnet50`, and `resnet101`) across two different classification topologies (`Linear` and `MLP`) and under two distinct fine-tuning regimes: full fine-tuning versus partial fine-tuning (restricting gradient updates to the deeper layers). The MLP architecture structured with two sequential hidden layers of dimensions $H_1 = 256$ and $H_2 = 128$, respectively. Leveraging the Weights & Biases platform, it was possible to track and analyze the real-time loss and accuracy curves throughout the entire training, validation, and testing pipelines.

2.3.1 Experimental Specifications

The optimization pipeline utilized the Adam optimizer paired with a Cross-Entropy loss function, configured with a learning rate of 1×10^{-5} and a batch size of 64. All input images were standardized to a spatial resolution of 64×64 pixels across 3 channels.

2.3.2 Training Phase

The validation set was generated via a stratified split of the original training dataset, allocating 80% of the samples for training and 20% for validation. All evaluated architectures converged to comparable final loss and accuracy metrics, with the primary distinction residing in their respective convergence trajectories.

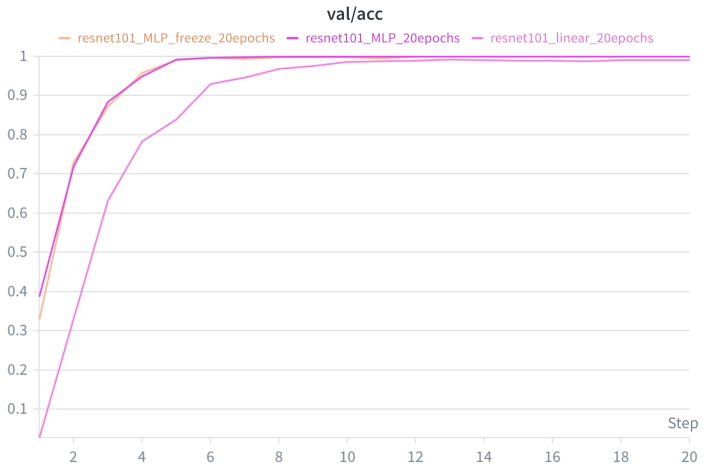

Figure 1: Validation accuracy trajectories for the `resnet101` backbone across different classifier configurations: MLP, Linear, and MLP with layer freezing.

As illustrated by the curves in Figure 1, the models equipped with an MLP head—both fully unfrozen and with layer freezing—exhibit a faster convergence rate. This behavior suggests that multi-layer perceptron configurations possess a superior capacity for capturing non-linear relationships compared to simple linear layers in this specific domain.

A detailed analysis of the MLP-based configurations in Figure 2 reveals that `resnet101` achieves the fastest convergence trajectory. This aligns with theoretical expectations, as the higher capacity and depth of the network facilitate accelerated pattern recognition during the early epochs of the fine-tuning process.

2.3.3 Testing

During the testing phase, the classification accuracy indicates that the optimal configuration consists of an MLP head combined with full fine-tuning of the model. Although the overall performance across all evaluated model combinations remains remarkably high, the linear classification head consistently yields a higher test loss. Restricting gradient updates by freezing the first layer of the model achieves comparable accuracy to the full fine-tuning regime while significantly reducing the computational overhead during training. Furthermore, the accuracy and loss were evaluated under more aggressive freezing strategies, specifically locking the first 2 and 3 layers; however, these configurations resulted in a sharp performance degradation compared to the previously observed cases. The empirical evidence demonstrating that the accuracy of models equipped with a `Linear` head is inversely proportional to the backbone's complexity con-

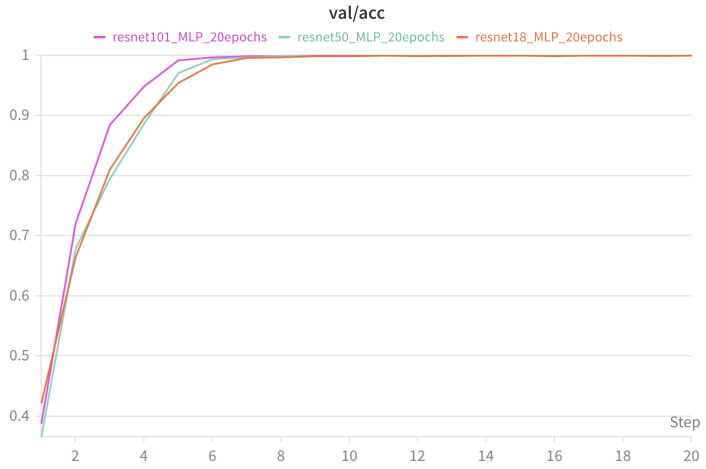

Figure 2: Validation accuracy comparison across different ResNet backbones utilizing the MLP classification head.

Classification Head	**Backbone**	**Freeze**	**Test Accuracy**	**Test Loss**
Linear	`resnet18`	No	**0.9618**	0.2250
	`resnet50`	No	0.9551	0.2674
	`resnet101`	No	0.9280	0.6376
MLP	`resnet18`	No	0.9675	0.1187
	`resnet50`	No	**0.9687**	0.1195
	`resnet101`	No	0.9611	0.1518
MLP	`resnet18`	Yes	0.9512	0.1790
	`resnet50`	Yes	**0.9623**	0.1392
	`resnet101`	Yes	0.9526	0.1814

Table 2: Test accuracy and test loss comparison across ResNet backbones, ordered by classification head configuration (Linear, MLP, and MLP with freezing).

firms that simple linear classifiers perform optimally only when operating on lower-dimensional, less abstracted feature spaces. Consequently, a single linear mapping proves incapable of effectively parsing the highly non-linear, deep feature representations extracted by a high-capacity model such as `resnet101`.

Conversely, the `MLP` head guarantees robust and stable performance across all evaluated backbones. This stability is directly attributable to the architectural integration of dense layers augmented by non-linear activation functions, which provide the necessary mathematical flexibility to successfully reconcile and map diverse latent representations onto the target category space.

3 Exercise 2: pipeline configuration

The project architecture ensures a clean abstraction of the core components of the deep learning workflow. This is achieved through a config.yaml configuration file utilized alongside the Hydra framework.

Why hydra? My high school computer science teacher used to say: "Don't use an atomic bomb to kill a fly." I am fully aware that, given the small scale of this project, Hydra might be an unnecessary complication. However, for educational purposes and the desire to learn a new tool, I decided to integrate it into at least one laboratory.

```
# config file
defaults:
  - model: resnet18
  - opt: adam
  - loss: cross_entropy
  - dataset: GTSRB
  - train: default
  - wandb: default
  - experiments: experiment_3_2
  - _self_
```

The project architecture allows for the abstraction of key deep learning components — specifically the model, optimizer, and loss function — decoupling them from the core software execution. This flexibility is achieved by leveraging Python dictionaries as a registry to map configuration strings to their respective classes or functions. Specific hyperparameters, such as learning rate and batch size, are encapsulated within Hydra's hierarchical configuration files dedicated to each module, ensuring a clean, modular, and centralized management.

```
# opt/adam.yaml
_target_: torch.optim.Adam
lr: 0.001
```

The project structure is organized into modular components to decouple execution logic from implementation details: `data_setup.py` manages DataLoader

construction based on the model configuration, `engine.py` defines the training, validation, and testing routines, and `utils.py` encapsulates auxiliary helper functions. This architectural design provides a high level of abstraction over specific hyperparameter settings and low-level implementations. Consequently, it enables seamless benchmarking and comparison across different model backbones and hyperparameter configurations.
Model monitoring during the training, validation, and testing phases is conducted utilizing Weights & Biases.

4 Exercise 3.2

4.1 Implementation

For feature extraction, the backbones of various models from the `torchvision` module were utilized, and their performances were benchmarked using multiple metrics. The classification head of each model was removed while keeping the remaining architecture intact; no fine-tuning was performed, resulting in a frozen feature extractor. Image feature similarity was computed on the GPU using **cosine similarity**. By sorting the similarity scores for each query of a given class *cls*, it is possible to retrieve the most similar elements from the training dataset and verify whether they belong to the same class.

4.2 Backbone evaluation

To provide a global evaluation metric capable of assessing the backbone's effectiveness in generating features suitable for information retrieval, a function implementing the mean Average Precision (mAP) across the dataset classes was defined. Specifically, the Average Precision (AP) is computed for each class present in the query data, and the mean is subsequently calculated across all class-wise precision scores. The resulting single value effectively quantifies the overall retrieval precision of the backbone across the entire dataset. The evalua-

Table 3: Mean Average Precision (mAP) performance across different backbone architectures.

Backbone	**mAP**
`inception_v3`	0.1871
`resnet18`	0.1774
`resnet101`	0.1692
`resnet50`	0.1690
`mobilenet_v2`	0.1545
`vgg19`	0.1412
`mobilenet_v3_small`	0.1298
`convnext_base`	0.1140

tion results reveal a significant counter-intuitive trend regarding backbone complexity. Counter to expectations, modern and highly complex architectures, such as `convnext_base`, yielded the poorest retrieval performances, whereas shallower networks like `resnet18` outperformed deeper variants (`resnet50` and `resnet101`). This phenomenon can be attributed to the nature of the frozen feature extractors: deep networks inherently distill high-level semantic features tailored to their pre-training domain (e.g. ImageNet), often discarding the low-level geometric and color-based features that are critical for distinguishing traffic signs. Furthermore, the superior performance of `inception_v3` highlights the effectiveness of multi-scale feature extraction in capturing both the global shape and the fine-grained symbolic details of the signs.
By analyzing the chart in figure 3 illustrating the highest and lowest class-wise Average Precision (AP) scores within the dataset, specific retrieval behaviors can be correlated with distinct data characteristics. Information retrieval achieves greater success in categories such as class 14 (representing STOP signs) or class 12 (representing priority road signs). Both of these classes feature unique geometric elements that differ significantly from those found in other categories. Conversely, the classes yielding the poorest information retrieval performance are primarily those associated signs classes geometrically and graphically similar to other classes. These categories exhibit highly similar visual patterns, typically differing only by the specific numerical digits displayed. Consequently, the feature extractor fails to assign any semantic meaning to these numbers, processing them strictly based on low-level visual characteristics.

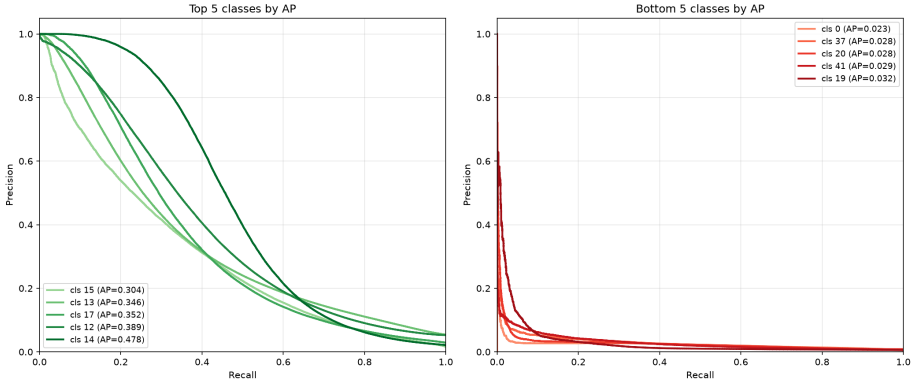

Figure 3: Best (left plot) and worst (right) class AP using resnet18 as backbone

4.3 Nearest mean classifier

Subsequently, a Nearest Mean Classifier (NMC) is implemented utilizing the best-performing backbone identified in the previous evaluation phase `inception_v3`. This classifier accepts the extracted features of a query image as input and determines its class membership based on feature proximity. Specifically, the clas-

sification is performed by computing the cosine similarity between the feature vector of the individual query image and the mean feature vector of each class, which is derived by averaging the features of all training samples belonging to that respective category.
Consequently, the classification accuracy of the NMC is evaluated and benchmarked across the different backbone feature extractors. The classification accuracy trends demonstrate strong alignment with the previously reported mAP scores, reinforcing the consistency between the models' information retrieval capabilities and their final classification performance.

Backbone	**Accuracy**
`inception_v3`	0.5005
`resnet18`	0.5005
`resnet101`	0.4750
`resnet50`	0.4595
`mobilenet_v2`	0.4565
`convnext_base`	0.4230
`vgg19`	0.4146
`mobilenet_v3_small`	0.3983

Table 4: NMC classification accuracy across different backbone architectures.